Look-alike Modelling

How FastIdentity turns a consented seed audience into a ranked look-alike — and where the ad platform takes over

This note documents the `domain.lookalike` model: what it learns from, how it scores, what it deliberately refuses to do, and how to run it. It is a **basic, auditable baseline** built on core scikit-learn, not a black box. The companion appendix in the platform guide is the one-screen summary.

1. The division of labour

“Look-alike” (Meta Lookalike, Google Similar/optimised audiences) is two jobs, and FastIdentity only owns the first:

- **Seed (FastIdentity).** Build a clean, *consented*, high-quality source audience and rank the rest of the **first-party** base by how much each profile resembles it. Only people already in the identity graph are ever scored — we never try to model or name strangers.
- **Expansion (the ad platform).** The population-level “find millions who resemble these” step happens inside Meta/Google, from *their* graph. FastIdentity hands the platform the best possible seed (hashed, consented) and keeps it fresh.

The score FastIdentity produces is therefore an **internal propensity/resemblance ranking** — useful on its own for prioritising owned-channel outreach, and as the seed selector for platform expansion.

2. What the model learns from

One feature vector per live profile (merged/erased profiles excluded), built **only** from first-party signals. Raw PII and raw identifiers (email, phone, device fingerprints, IP) are **never** features.

Group	Features	Why
Identity	`known`	a resolved person behaves differently from a one-session anon
Recency / tenure	`recency_days`, `tenure_days`	fresh, sustained interest resembles a buyer
Behaviour	`log_events`, `event_types`, `ev=<event>` (incl. `product_viewed`, `booking_started`, `order_completed`)	stage and depth of engagement
Interests	`n_interest_keys`, `interest_weight`, `int=<key>:<value>` (e.g. `int=treatment:hip replacement`)	*what* the person is actually interested in — the strongest signal for a medical marketplace
Devices	`n_devices`, `dev=mobile|desktop|tablet|tv`	device mix correlates with intent and channel
Household	`in_household`, `household_size`	multi-person research is a different shape of demand

Categorical interest/event indicators are sparse (`DictVectorizer`), so the vocabulary grows with the data without any manual schema. **Consent is reported per profile but is never a feature** — it is an activation gate, not a similarity signal, and letting it leak into the model would just learn “who we are allowed to email”, not “who resembles a buyer”.

3. The model

The seed is positive-only; we have no labelled “non-look-alikes”. This is a textbook **PU-learning** (positive–unlabelled) problem, and we use its simplest honest baseline:

1. **Label.** Seed profiles are the positive class (`y = 1`). A random sample of the non-seed base is drawn as negatives (`y = 0`) — the “unlabelled-as-negative” assumption. The sample size is `LOOKALIKE_NEG_RATIO` (default 3) negatives per positive, floored at 50, capped at the population.
2. **Vectorise & scale.** `DictVectorizer` → sparse matrix; `MaxAbsScaler` (sparse-safe, no mean centering) puts every feature on a comparable scale.
3. **Classify.** `LogisticRegression(max_iter=1000, class_weight="balanced", random_state=42)`. Balanced class weights offset the deliberate negative over-sampling; logistic regression gives a calibrated, **inspectable** coefficient per feature — you can read *why* a profile scored high.
4. **Score.** The predicted probability `P(class = 1)` for each non-seed profile is its look-alike score in `[0, 1]`. Seed members are excluded from the output (they are already in the audience).

Because unlabelled negatives include some true positives, the absolute probabilities are a monotone proxy rather than a calibrated base rate — which is exactly what a **ranking** needs. The whole pipeline is deterministic (fixed `random_state` and sample seed), so the same inputs give the same audience.

4. Outputs

`score_lookalike(...)` returns:

- `ok`, `engine`, `seed_size`, `scored`
- `train_auc` — ROC-AUC of the fitted model on its own training set (a sanity check that the seed is separable from the sampled base; not a generalisation estimate)
- `top_features` — the highest positive logistic-regression coefficients, so each run is explainable
- `audience` — non-seed profiles ranked by score (highest first), each with its `status` and consent flags, honouring `limit` and an optional `min_score`

`lookalike_for_segment(tenant, segment_id, ...)` is the usual entry point: it takes a saved segment's members as the seed, so you seed on **bookers (last 90 days)**, **high-intent viewers** or a **high-value specialty** without hand-listing ids.

5. Guardrails

- **Seed floor.** Below `LOOKALIKE_MIN_SEED` (default 25) the model refuses to run and says so — a tiny seed overfits and, downstream, platforms need ~100+ matched people before expansion is reliable. This is why growing the known base (the identification ladder) directly improves look-alike quality.
- **Seed on behaviour and stage, never on diagnosis.** A seed such as “booked orthopaedics” is a business segment; it must never become health-condition targeting of an individual. Keep seed names and their use at the destination/stage level.
- **Consent is the gate, not the model.** Only marketing-consented, SHA-256-hashed identifiers are synced to a platform as the seed; the audience rows carry consent flags so activation can filter before any sync. Converters and opt-outs are excluded (and uploaded as suppression/exclusion audiences).
- **First-party only.** Strangers are never modelled; nothing leaves FastIdentity from this module — activation still passes the consent, suppression and frequency gates.

6. Engines, configuration & operating notes

Two engines, mirroring `domain.matching`:

- **`sklearn`** (default) — the logistic-regression model above. Needs `pip install -r requirements-lookalike.txt`.
- **`builtin`** — a dependency-free fallback: L2-normalise each feature vector, take the seed centroid, score candidates by cosine similarity to it. No training, no probabilities — a resemblance ranking that keeps the feature usable where scikit-learn is not installed. The module degrades to this automatically if `sklearn` is requested but unavailable.

Setting	Default	Meaning
`LOOKALIKE_ENGINE`	`sklearn`	`sklearn` or `builtin`
`LOOKALIKE_MIN_SEED`	`25`	smallest seed that will be modelled
`LOOKALIKE_NEG_RATIO`	`3`	sampled negatives per seed positive
`LOOKALIKE_DEFAULT_LIMIT`	`500`	max audience rows returned

Operating cadence. Re-score on a schedule so the audience tracks current bookers; stale seeds drift. Watch `train_auc` — a sudden drop means the seed is no longer separable (seed too broad, or a data gap). Spot-check `top_features` so the ranking leans on interests and intent, not an artefact.

```
from domain.lookalike import lookalike_for_segment
res = lookalike_for_segment("demo", bookers_segment_id, limit=1000)
# res["audience"] → consented, ranked look-alike seed to hash + sync to Meta/Google
```